



INFORMATIQUE

Sequence 6 : Types structurés

TD 6.1

1h30 -
v2025-11-
17 *IUT d'Annecy, 9 rue de l'Arc en Ciel, 74940 Annecy*

INTRODUCTION AUX TYPES STRUCTURÉS

Exercice 1 : Le problème sans structure

On souhaite gérer les informations d'un étudiant :

- son prénom,
- son âge,
- sa moyenne.

Question 1 Proposer une liste de variables pour stocker ces informations.

Par exemple :

```
string prenom;  
int age;  
float moyenne;
```

On peut discuter de la taille du tableau de caractères (ici 30).

Question 2 Écrire un code qui définit puis affiche les informations d'une étudiante nommée « Alice », 20 ans, moyenne 14.2.

```
string prenom = "Alice";  
int age = 20;  
float moyenne = 14.2f;  
  
5 printf("%s, %d ans, moyenne = %.1f\\n", prenom, age, moyenne);
```

Question 3 Comment stocker les informations de 3 étudiants différents ?

Sans structure, on peut multiplier les variables :

```
string prenom1, prenom2, prenom3;  
int age1, age2, age3;  
float moyenne1, moyenne2, moyenne3;
```

On peut aussi utiliser 3 tableaux parallèles :

```
string prenom[3];  
int ages[3];  
float moyennes[3];
```

Mais on voit déjà que cela devient plus difficile à suivre.

Question 4 Pourquoi cette approche devient-elle source d'erreurs ?

*Parce qu'il faut constamment **se souvenir** que les éléments `exttprenom1`, `age1`, `moyenne1` vont ensemble, etc. On risque facilement de mélanger les indices ou d'oublier un champ. On ne manipule jamais « un*

étudiant » mais « plusieurs variables séparées », ce qui augmente la probabilité d'erreurs et rend le code difficile à maintenir.



Les types structurés en C

En C, on peut regrouper plusieurs variables dans une seule entité appelée **structure**, définie avec le mot-clé **struct**.

Dans notre cas, nous allons créer un type structuré qui contiendra différentes informations concernant une même entité.

La définition d'un type structuré se fait généralement avec le mot-clé **typedef** pour simplifier son utilisation ultérieure. La syntaxe générale est la suivante :

```
typedef struct {  
    type1 champ1;  
    type2 champ2;  
    ...  
5    typeN champN;  
} NomDeLaStructure;
```

Prenons l'exemple d'un contact dans un carnet d'adresses. On peut définir une structure **Contact** qui regroupe plusieurs champs :

```
typedef struct {  
    string nom;  
    string prenom;  
    string telephone;  
    string email;  
5    int anneeNaissance;  
} T_Contact;
```

Exercice 2 : Créer une structure cohérente

On regroupe les données dans une seule entité : un **type structuré**.

Question 1 À l'aide de l'exemple précédent, écris la définition d'une structure **Etudiant** contenant les champs correspondants aux informations d'un étudiant (prénom, âge, moyenne).

*Par exemple, avec un **typedef** :*

```
typedef struct {  
    string prenom;  
    int age;  
    float moyenne;  
5 } T_Etudiant;
```

Question 2 Écrire les déclarations suivantes :

- une variable **etudiant_1** représentant un étudiant,
- un tableau **classe** de 30 étudiants.

```
T_Etudiant etudiant_1;  
T_Etudiant classe[30];
```

Question 3 Écrire les lignes de code permettant d'initialiser les informations de l'étudiant **etudiant_1** avec les valeurs suivantes : prénom « Bob », âge 22, moyenne 15.5.

Deux possibilités :

Initialisation directe :

```
| T_Etudiant etudiant_1 = {"Bob", 22, 15.5f};
```

Affectation champ par champ :

```
| etudiant_1.prenom = "Bob";
| etudiant_1.age = 22;
| etudiant_1.moyenne = 15.5f;
```

Question 4 Écrire les lignes de code permettant de demander à l'utilisateur de saisir les informations des 30 étudiants du tableau `classe`.

```
5 | int i;
   | for (i = 0; i < 30; i++) {
   |     printf("Etudiant %d\\n", i+1);
   |     classe[i].prenom = get_string("Prenom : ");
   |
   |     classe[i].age = get_int("Age : ");
   |
   |     classe[i].moyenne = get_float("Moyenne : ");
10 | }
```

Question 5 Écrire les lignes de code permettant d'afficher le prénom et la moyenne du meilleur étudiant de la classe.

```
5 | int i, indice_meilleur = 0;
   | for (i = 1; i < 30; i++) {
   |     if (classe[i].moyenne > classe[indice_meilleur].moyenne) {
   |         indice_meilleur = i;
   |     }
   | }
10 | printf("Meilleur etudiant : %s, moyenne = %.2f\\n",
   |     classe[indice_meilleur].prenom,
   |     classe[indice_meilleur].moyenne);
```

Exercice 3 : Représenter une heure

On veut manipuler une heure sous la forme HH:MM:SS.

Question 1 Proposer un type structuré `T_Heure` permettant de stocker les informations d'un horaire.

```
5 | typedef struct {
   |     int h;
   |     int m;
   |     int s;
   | } T_Heure;
```

Question 2 Écrire une fonction `void afficher_heure(T_Heure t)` qui affiche HH:MM:SS.

```
| void afficher_heure(T_Heure t) {
|     printf("%02d:%02d:%02d\\n", t.h, t.m, t.s);
| }
```

Question 3 Écrire `int en_secondes(T_Heure t)` qui renvoie le nombre total de secondes.

```
int en_secondes(T_Heure t) {  
    return t.h * 3600 + t.m * 60 + t.s;  
}
```

Question 4 Créer deux horaires dans le `main` et afficher la différence en secondes.

```
int main(void) {  
    T_Heure h1 = {8, 30, 0};  
    T_Heure h2 = {10, 15, 30};  
  
5    int s1 = en_secondes(h1);  
    int s2 = en_secondes(h2);  
  
    printf("Différence = %d secondes\n", s2 - s1);  
    return 0;  
10 }
```

Question 5 Ecrire la fonction inverse qui prend en paramètre un entier représentant un nombre de secondes et qui renvoie une structure `T_Heure` correspondante.

```
T_Heure inverse(int total_secondes) {  
    T_Heure t;  
    t.h = total_secondes / 3600;  
    total_secondes %= 3600;  
5    t.m = total_secondes / 60;  
    t.s = total_secondes % 60;  
    return t;  
}
```

Exercice 4 : Gérer un produit en stock

On veut garder les informations suivantes pour chaque produit :

- son nom (chaîne de caractères),
- son prix HT (nombre à virgule),
- son taux de TVA (nombre à virgule).

Question 1 Proposer un type structuré `T_Produit` pour stocker ces informations.

```
typedef struct {  
    string nom;  
    float prixHT;  
    float tva; // en pourcentage, par ex. 20.0  
5 } T_Produit;
```

Question 2 Écrire une fonction qui renvoie le prix TTC d'un produit (la fonction ne prendra qu'un seul argument).

```
float prixTTC(T_Produit p) {  
    return p.prixHT * (1.0f + p.tva / 100.0f);  
}
```

On pourra ensuite utiliser cette fonction dans un `main` pour comparer plusieurs produits.

Exercice 5 : Gestion d'un vecteur 2D

On manipule des vecteurs dans un plan :

```
typedef struct{  
    float x;  
    float y;  
} T_Vecteur;
```

Question 1 Écrire une fonction qui renvoie la norme $\sqrt{x^2 + y^2}$ d'un vecteur donné en paramètre.

```
#include <math.h>  
  
float norme(T_Vecteur v) {  
    return sqrtf(v.x * v.x + v.y * v.y);  
5 }
```

Question 2 Écrire une fonction permettant de sommer deux vecteurs.

```
T_Vecteur addition(T_Vecteur a, T_Vecteur b) {  
    T_Vecteur res;  
    res.x = a.x + b.x;  
    res.y = a.y + b.y;  
5    return res;  
}
```

Question 3 Écrire une fonction permettant de donner l'opposé d'un vecteur.

```
T_Vecteur oppose(T_Vecteur v) {  
    T_Vecteur res;  
    res.x = -v.x;  
    res.y = -v.y;  
5    return res;  
}
```

Question 4 Dans le main, déclarer trois vecteurs, afficher leurs normes, puis afficher la norme de $a - b$.

```
int main(void) {  
    T_Vecteur a = {3.0f, 4.0f};  
    T_Vecteur b = {1.0f, 2.0f};  
    T_Vecteur c = {-2.0f, 0.5f};  
5  
    printf("||a|| = %f\n", norme(a));  
    printf("||b|| = %f\n", norme(b));  
    printf("||c|| = %f\n", norme(c));  
  
10    T_Vecteur diff;  
    diff.x = a.x - b.x;  
    diff.y = a.y - b.y;  
  
    printf("||a-b|| = %f\n", norme(diff));  
15    return 0;  
}
```

Exercice 6 : Extension : struct imbriquées

Il est également possible de créer des structures imbriquées, c'est-à-dire d'utiliser une structure comme champ d'une autre structure. Par exemple, on peut définir une structure **Adresse** et l'utiliser dans une structure **Etudiant** :

```
typedef struct {
    string rue;
    int codePostal;
    string ville;
5 } T_Adresse;

typedef struct {
    string prenom;
    int age;
10 float moyenne;
    T_Adresse adresse;
} T_EtudiantAdresse;
```

Question 1 Comment accéder à la ville d'un étudiant ?

*Si **e** est une variable de type **T_EtudiantAdresse**, on accède à la ville par :*

```
| e.adresse.ville
```

Question 2 Quel est l'avantage de créer une structure imbriquée plutôt qu'ajouter 3 champs à **Etudiant** ?

*On regroupe tous les champs liés à l'adresse dans une seule structure **T_Adresse**. Cela permet :*

- de réutiliser **T_Adresse** pour d'autres structures (*Professeur, Entreprise, etc.*);
- de rendre le code plus lisible (une entité logique = une structure);
- de faciliter les modifications (ajouter un champ à l'adresse sans toucher toutes les structures qui l'utilisent).

Question 3 Proposer une fonction `int habite_ville(T_EtudiantAdresse e, const char* ville)` qui renvoie 1 si l'étudiant habite dans la ville donnée, 0 sinon.

```
| int habite_ville(T_EtudiantAdresse e, const char* ville) {
|     return (strcmp(e.adresse.ville, ville) == 0);
| }
```

On peut ensuite parcourir un tableau d'étudiants et afficher ceux qui habitent dans une ville donnée.

Exercice 7 : Projet d'ensemble : bulletins d'une classe

On souhaite écrire un petit programme qui gère les **bulletins** d'une classe. Chaque élève a plusieurs notes dans différentes matières, avec un **coefficient** pour chaque matière.

On suppose que :

- tous les élèves ont les **mêmes matières**, dans le même ordre,
- il y a au maximum 30 élèves dans la classe,
- il y a par exemple 4 matières par élève,
- chaque élève peut avoir **plusieurs notes** par matière (par ex. contrôles, devoirs, etc.).

Question 1 Proposer des constantes et des types structurés permettant de modéliser ce problème.

Par exemple :

```
| #define TAILLE_MAX_CLASSE 30
| #define NB_MATIERES      4
| #define NB_NOTES_MAX     10
5 typedef struct {
|     string nom;
|     float coeff;
```

```

    } T_Matiere;

10 typedef struct {
    float notes[NB_MATIERES][NB_NOTES_MAX];
    int nb_notes[NB_MATIERES]; // nombre de notes pour chaque matière
    string prenom;
    int age;
15 } T_Eleve;

    T_Matiere matieres[NB_MATIERES];

```

Cette approche sépare les définitions des matières (nom et coefficient, partagés par tous) du stockage des notes individuelles de chaque élève. Chaque élève peut avoir un nombre différent de notes par matière, stocké dans `nb_notes[]`.

Question 2 Écrire une fonction `float moyenne_eleve(T_Eleve e, T_Matiere matieres[])` qui renvoie la moyenne générale de l'élève, en tenant compte des coefficients de chaque matière.

```

float moyenne_eleve(T_Eleve e, T_Matiere matieres[]) {
    int i, k;
    float somme_notes = 0.0f;
    float somme_coeffs = 0.0f;

5     for (i = 0; i < NB_MATIERES; i++) {
        float moy_matiere = 0.0f;

        // Calculer la moyenne pour cette matiere
10        for (k = 0; k < e.nb_notes[i]; k++) {
            moy_matiere += e.notes[i][k];
        }
        if (e.nb_notes[i] > 0) {
            moy_matiere /= e.nb_notes[i];
15        }

        // Ajouter à la moyenne pondérée
        somme_notes += moy_matiere * matieres[i].coeff;
        somme_coeffs += matieres[i].coeff;
20    }

    if (somme_coeffs == 0.0f) {
        return 0.0f;
    }

25    return somme_notes / somme_coeffs;
}

```

Pour chaque matière, on calcule d'abord la moyenne de toutes les notes, puis on l'intègre dans la moyenne générale pondérée par le coefficient.

Question 3 Écrire une fonction `float moyenne_matiere(T_Eleve classe[], int nbEleves, int indiceMatiere)` qui renvoie la moyenne de la classe pour une matière donnée (identifiée par son indice).

```

float moyenne_matiere(T_Eleve classe[], int nbEleves, int indiceMatiere) {
    int i, k;
    float somme = 0.0f;
    int nb_notes_total = 0;

5     for (i = 0; i < nbEleves; i++) {
        for (k = 0; k < classe[i].nb_notes[indiceMatiere]; k++) {
            somme += classe[i].notes[indiceMatiere][k];

```

```

10         nb_notes_total++;
    }
}

    if (nb_notes_total == 0) {
        return 0.0f;
15    }

    return somme / nb_notes_total;
}

```

On parcourt tous les élèves et toutes leurs notes pour la matière donnée, en tenant compte du nombre réel de notes saisies pour chaque élève.

Question 4 On va maintenant écrire une partie de main. On dispose d'un tableau T_Eleve classe[TAILLE_MAX_CLASSE]; et d'un tableau T_Matiere matieres[NB_MATIERES];. Écrire le code permettant :

1. de demander à l'utilisateur combien d'élèves sont dans la classe;
2. de saisir les matières (nom et coefficient) une seule fois;
3. de saisir pour chaque élève : son prénom, son âge, puis ses notes dans les matières;
4. d'afficher, pour chaque élève, sa moyenne générale.

```

int main(void) {
    T_Eleve classe[TAILLE_MAX_CLASSE];
    T_Matiere matieres[NB_MATIERES];
    int nbEleves;
5    int i, j, k;

    // 1. Saisie du nombre d'eleves
    do {
        nbEleves = get_int("Nombre d'eleves (1..%d) : ", TAILLE_MAX_CLASSE);
10    } while (nbEleves < 1 || nbEleves > TAILLE_MAX_CLASSE);

    // 2. Saisie des matieres
    for (j = 0; j < NB_MATIERES; j++) {
        printf("Matiere %d\n", j + 1);
15        matieres[j].nom = get_string("Nom de la matiere : ");
        matieres[j].coeff = get_float("Coefficient : ");
    }
    printf("\n");

20    // 3. Saisie des eleves et de leurs notes
    for (i = 0; i < nbEleves; i++) {
        printf("Eleve %d\n", i + 1);
        classe[i].prenom = get_string("Prenom : ");
        classe[i].age = get_int("Age : ");
25

        for (j = 0; j < NB_MATIERES; j++) {
            classe[i].nb_notes[j] = get_int("Nombre de notes en %s : ",
                                                matieres[j].nom);

            for (k = 0; k < classe[i].nb_notes[j]; k++) {
30                classe[i].notes[j][k] = get_float(
                    "Note %d : ", k + 1);
            }
        }
        printf("\n");
35    }

    // 4. Affichage des moyennes
    for (i = 0; i < nbEleves; i++) {
40        float m = moyenne_eleve(classe[i], matieres);
    }
}

```



```

        printf("Moyenne de %s = %.2f\\n", classe[i].prenom, m);
    }

    return 0;
45 }

```

Pour chaque matière et chaque élève, on demande d'abord le nombre de notes, puis les notes individuelles. Cela permet une flexibilité complète.

Question 5 À partir des fonctions précédentes, écrire le code (dans le `main` ou dans une fonction dédiée) qui :

- détermine l'élève ayant la meilleure moyenne générale et affiche son prénom et sa moyenne ;
- calcule la moyenne de la classe pour chaque matière ;
- affiche la matière dont la moyenne de classe est la plus faible (matière « à renforcer »).

Recherche du meilleur élève :

```

int indice_meilleur = 0;
float meilleure_moy = moyenne_eleve(classe[0], matieres);
float m;
int i;

5
for (i = 1; i < nbEleves; i++) {
    m = moyenne_eleve(classe[i], matieres);
    if (m > meilleure_moy) {
        meilleure_moy = m;
10        indice_meilleur = i;
    }
}

printf("Meilleur eleve : %s (moyenne = %.2f)\\n",
15     classe[indice_meilleur].prenom, meilleure_moy);

```

Moyennes par matière et matière la plus faible :

```

float moyennesMatiere[NB_MATIERES];
int j;

for (j = 0; j < NB_MATIERES; j++) {
5     moyennesMatiere[j] = moyenne_matiere(classe, nbEleves, j);
    printf("Moyenne de la classe en %s : %.2f\\n",
        matieres[j].nom,
        moyennesMatiere[j]);
}

10
// Recherche de la matiere avec la plus faible moyenne
int indice_pire_matiere = 0;
float pire_moyenne = moyennesMatiere[0];

15
for (j = 1; j < NB_MATIERES; j++) {
    if (moyennesMatiere[j] < pire_moyenne) {
        pire_moyenne = moyennesMatiere[j];
        indice_pire_matiere = j;
    }
20 }

printf("Matiere a renforcer : %s (moyenne = %.2f)\\n",
    matieres[indice_pire_matiere].nom,
    pire_moyenne);
25

```

Cet ensemble de questions mobilise :

- plusieurs **structures** (struct avec données partagées et individuelles),
- des **tableaux** de structures,

- des **fonctions** qui prennent et renvoient des structures,
- des **boucles imbriquées** et un peu d'algorithmique (recherche de max/min).